
KHYBER PAKHTUNKHWA BOARD OF TECHNICAL AND COMMERCE EDUCATION PESHAWAR



Government College of Technology Peshawar

PESHAWAR

DIPLOMA IN INFORMATION TECHNOLOGY (DIT)

FINAL SEMESTER PROJECT

Student Result Management System

SUBMITTED BY

Omaina Rizwan

Khadija BiBi

ROLL NO.

239083

239095

REGISTRATION

GCT/P/DIT/1st-25/E-8647

GCT/P/DIT/Ist-25/E-8790

ACADEMIC SESSION

2026

Approval / Certificate

This is to certify that the project report entitled “**Student Result Management System (SRMS)**” has been prepared as a final-semester project for the Diploma in Information Technology. The work is submitted for review and approval according to the academic requirements of the institute.

Declaration

I declare that this project report, titled **Student Result Management System (SRMS)**, has been prepared for my Diploma in Information Technology final-semester project. The report describes the implementation that exists in the submitted project. Wherever project data is shown, it is identified as demonstration or test data. No backend, cloud database, or unimplemented feature is presented as part of the current system.

Student Signatures:

Omaima Rizwan:_____

Khadija BiBi:_____

Date: 11/9/2026

Dedication

This project is dedicated to my parents, teachers, and everyone who encouraged me to continue learning information technology and complete this final-semester work.

Acknowledgement

I am thankful to Allah for giving me the strength and patience to complete this project. I am grateful to my teachers and supervisor for their guidance, suggestions, and encouragement throughout the development process. I also thank my institute for providing the learning environment required for this work. Finally, I appreciate the support of my family and friends during the planning, implementation, testing, and documentation of the Student Result Management System.

Abstract

Educational institutions need organized student and examination records so that academic information can be entered, reviewed, updated, and presented accurately. Manual registers and separate paper calculations make it difficult to locate records, update information, and prepare result summaries. This project presents a **Student Result Management System (SRMS)** developed as a frontend-only Diploma in Information Technology project. The system provides a simple browser-based workspace for an administrator to manage student records and examination results.

The application is implemented with HTML5, CSS3, and Vanilla JavaScript. Browser LocalStorage is used to keep student, result, and settings data as JSON records. The system includes a welcome and demonstration-login page, dashboard statistics, a student directory, add/edit student forms, result management, search and filtering, a live result summary, a printable result-card view, and responsive layouts for desktop and mobile screens. Five subjects are used in the result form: English, Mathematics, Computer, Physics, and Urdu. The application calculates total marks out of 500, percentage, grade, and PASS/FAIL status. A result passes when the percentage is at least 50% and every subject mark is at least 40. The dashboard calculates current student and result counts, pass and fail counts, pass percentage, average percentage, recent records, and grade distribution from LocalStorage.

Testing was performed through browser interaction, structural checks, persistence checks, responsive snapshots, and temporary CRUD records that were removed after verification. The clean demonstration dataset contains six students and six results, with five passed results, one failed result, an 83.33% pass percentage, and a 73.90% average percentage. The system demonstrates practical frontend development skills, but it is not a production information system. It has no server-side database, secure authentication, multi-user access, cloud synchronization, email delivery, or built-in PDF generation. These limitations provide clear directions for future enhancement.

Table of Contents

1. Chapter 1 — Introduction
2. Chapter 2 — System Study and Requirements Analysis
3. Chapter 3 — System Analysis and Design
4. Chapter 4 — Technologies Used
5. Chapter 5 — Implementation
6. Chapter 6 — Source Code Explanation
7. Chapter 7 — User Interface and Screenshots
8. Chapter 8 — Testing and Validation
9. Chapter 9 — Results and Discussion
10. Chapter 10 — Security, Limitations and Future Enhancements
11. References

List of Figures

Figure	Title
Figure 1	Welcome and login page
Figure 2	Dashboard overview
Figure 3	Student directory
Figure 4	Add Student interface
Figure 5	Results directory
Figure 6	Add Result interface
Figure 7	Result card and print view
Figure 8	Search Result interface
Figure 9	Mobile dashboard
Figure 10	Mobile student directory
Figure 11	LocalStorage source code
Figure 12	Student validation source code
Figure 13	Student search source code
Figure 14	Result calculation source code
Figure 15	Result CRUD source code
Figure 16	Dashboard statistics source code
Figure 17	Grade-distribution source code
Figure 18	System architecture
Figure 19	Use-case diagram
Figure 20	Data-flow diagram
Figure 21	Student-management flowchart
Figure 22	Result-management flowchart
Figure 23	Student-Result relationship

List of Tables

Table	Title
Table 1	Functional requirements
Table 2	Non-functional requirements
Table 3	Hardware requirements
Table 4	Software requirements
Table 5	Student data fields
Table 6	Result data fields
Table 7	Grade system
Table 8	Validation rules
Table 9	Test cases
Table 10	Boundary tests
Table 11	Current limitations and future enhancements

List of Abbreviations

Abbreviation	Meaning
SRMS	Student Result Management System
DIT	Diploma in Information Technology
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
CRUD	Create, Read, Update, Delete
UI	User Interface
UX	User Experience
API	Application Programming Interface
JSON	JavaScript Object Notation

Chapter 1 — Introduction

1.1 Introduction

A Student Result Management System is an application used to organize student profiles and examination records. It helps an administrator keep identity information, enter marks, calculate academic outcomes, and view or print result summaries. This project develops a small, understandable SRMS suitable for a DIT final-semester project.

1.2 Background

Student information and examination results are commonly maintained using registers, spreadsheets, or separate documents. These approaches can work for a small number of records, but they become difficult to search and update as the number of students grows. A browser-based interface can place the common actions in one workspace and can reduce repeated manual calculations.

1.3 Problem Statement

Manual result management may require repeated writing, manual addition of marks, and searching through paper or unrelated files. It can also make updates difficult because a change may need to be copied to several places. Preparing a result card takes additional time when totals, percentages, grades, and status are calculated manually. These are practical record-management problems, not claims that every institution experiences the same level of difficulty.

1.4 Proposed Solution

The proposed solution is a frontend-only SRMS that stores JSON records in the browser. The administrator can manage students, enter results for five subjects, use search and filters, view dashboard analytics, open a result card, and invoke the browser print function. The project keeps the implementation simple so that each module can be understood by a DIT student.

1.5 Aim

The aim is to develop a simple and usable Student Result Management System that demonstrates student CRUD, result CRUD, automatic academic calculations, browser-based persistence, searching, filtering, dashboard analytics, and printable result presentation.

1.6 Objectives

The project objectives are to manage student records; store student information in LocalStorage; search and filter students; edit and delete students; enter examination results; validate marks; calculate total marks and percentage; assign grades; determine PASS/FAIL; show dashboard statistics; display a result card; and provide a browser print action.

1.7 Scope

The current scope includes a demonstration login screen, administrator-style navigation, student profiles, student directory actions, five-subject result entry, result calculations, result search and filters, dashboard analytics, responsive layouts, and print styling. It is intended for local browser demonstration rather than institutional deployment.

1.8 Limitations

The project is frontend-only. It has no server-side database, secure production authentication, multi-user environment, role-based access control, cloud synchronization, email delivery, or built-in PDF-generation feature. Data is stored in the browser and can be removed or changed by the browser user. The visible login uses the fixed demonstration credentials `admin` and `admin123`; it is not secure authentication. The Print Result control invokes the browser print dialog rather than generating a PDF file inside the application.

1.9 Significance

The project demonstrates practical skills in semantic HTML, CSS layout, responsive design, DOM manipulation, event handling, validation, JSON serialization, LocalStorage persistence, CRUD workflows, calculations, and testing. It provides a complete small application that can be explained without relying on a complex framework.

1.10 Target User

The intended user is an administrator or operator who maintains student and examination records. The current demonstration does not include separate teacher, student, or administrator accounts.

1.11 Chapter Organization

Chapter 2 explains the system study and requirements. Chapter 3 describes architecture, modules, processes, and data design. Chapter 4 explains the technologies. Chapter 5 documents the implementation. Chapter 6 explains important source code. Chapter 7 presents genuine application screenshots. Chapter 8 records testing and validation. Chapter 9 discusses the final outcome. Chapter 10 explains security, limitations, and future work. Chapter 11 concludes the report.

Chapter 2 — System Study and Requirements Analysis

2.1 Existing System

The existing approach considered in this project is manual or semi-manual record management using paper files or separate basic documents. A user may enter student information in one place and calculate results in another place. This separation can make searching, updating, and checking results slower.

2.2 Problems in the Existing System

The main problems are repeated data entry, difficulty locating student records, possible arithmetic mistakes, time-consuming record maintenance, and repeated work when preparing result summaries. The SRMS addresses these problems within the limited scope of a local browser application.

2.3 Proposed System

The proposed SRMS combines student records, result records, calculations, dashboard summaries, and result-card presentation in linked pages. A result stores a `studentId`, allowing the application to resolve the student name and identity information when it renders a result directory or result card.

2.4 Feasibility Study

2.4.1 Technical Feasibility

HTML5, CSS3, and Vanilla JavaScript are suitable for this project because they are directly supported by modern browsers and are appropriate for demonstrating fundamental web-development skills. LocalStorage provides simple key/value persistence for a small educational dataset. The project does not require a server during normal browser use.

2.4.2 Economic Feasibility

The implementation uses freely available web technologies and does not require paid software, a hosted database, or special infrastructure for demonstration. The main cost is the developer's time for design, implementation, testing, and documentation.

2.4.3 Operational Feasibility

The interface uses a fixed sidebar, clear page headings, forms, tables, cards, badges, and action links. These patterns make the common tasks understandable for an administrator with basic computer knowledge.

2.5 Functional Requirements

ID	Requirement	Implemented behavior
FR-01	Access the application	Welcome page provides the

		demonstration login.
FR-02	Add a student	Valid student form data is saved to LocalStorage.
FR-03	Edit a student	Existing values are populated and identity numbers remain protected.
FR-04	Delete a student	A confirmation is requested before deletion.
FR-05	Search students	Search matches student name, roll number, or Student ID.
FR-06	Filter students	Class and section selectors combine with search.
FR-07	Add results	User selects a student, session, and five subject marks.
FR-08	Edit results	Existing result values are loaded and updated in place.
FR-09	Delete results	A confirmation is requested before deletion.
FR-10	Calculate total marks	Five subject marks are summed out of 500.
FR-11	Calculate percentage	Obtained marks are divided by 500 and multiplied by 100.
FR-12	Assign grade	The grade bands A+, A, B, C, D, and F are applied.
FR-13	Determine status	PASS requires percentage ≥ 50 and every subject ≥ 40 .
FR-14	Display dashboard statistics	Counts, percentages, average, recent records, and grade bars are dynamic.
FR-15	View result card	A selected result is rendered with student and subject details.
FR-16	Print result	The browser print function is invoked from the result-card page.

2.6 Non-Functional Requirements

Requirement	Application interpretation
Usability	Clear labels, grouped forms, readable tables, and direct actions.
Performance	Local records are read synchronously without network requests.
Maintainability	Logic is separated into storage, student, result, dashboard, and shared-app modules.
Responsiveness	Sidebar, cards, forms, and tables adapt to desktop, tablet, and mobile widths.
Accessibility	Labels, button types, focus behavior, ARIA labels, and empty states are used.
Reliability	Validation, duplicate checks, and safe empty states prevent common invalid records.
Simplicity	The project uses understandable HTML, CSS, and Vanilla JavaScript.

2.7 Hardware Requirements

Item	Minimum practical requirement
Processor	Dual-core processor or equivalent
Memory	4 GB RAM
Storage	Approximately 100 MB for source files and browser cache
Display	1366×768 or similar; mobile layouts are also supported
Input	Keyboard and pointing device or touchscreen

2.8 Software Requirements

Software	Use
Operating system	Any modern desktop or mobile operating system with a browser
Web browser	Modern Chromium, Firefox, Safari, or equivalent browser
Code editor	Any editor capable of editing HTML, CSS, and JavaScript
HTML5	Page structure and forms
CSS3	Layout, visual design, responsive rules, and print styling
Vanilla JavaScript	Application logic and DOM behavior

Chapter 3 — System Analysis and Design

3.1 System Architecture

The SRMS follows a browser-based frontend architecture. HTML5 provides the page structure, CSS3 controls presentation and responsive behavior, and JavaScript handles events, validation, data access, calculations, and rendering. The storage module uses LocalStorage keys named 'students', 'results', and 'settings'. There is no backend layer in the current implementation.

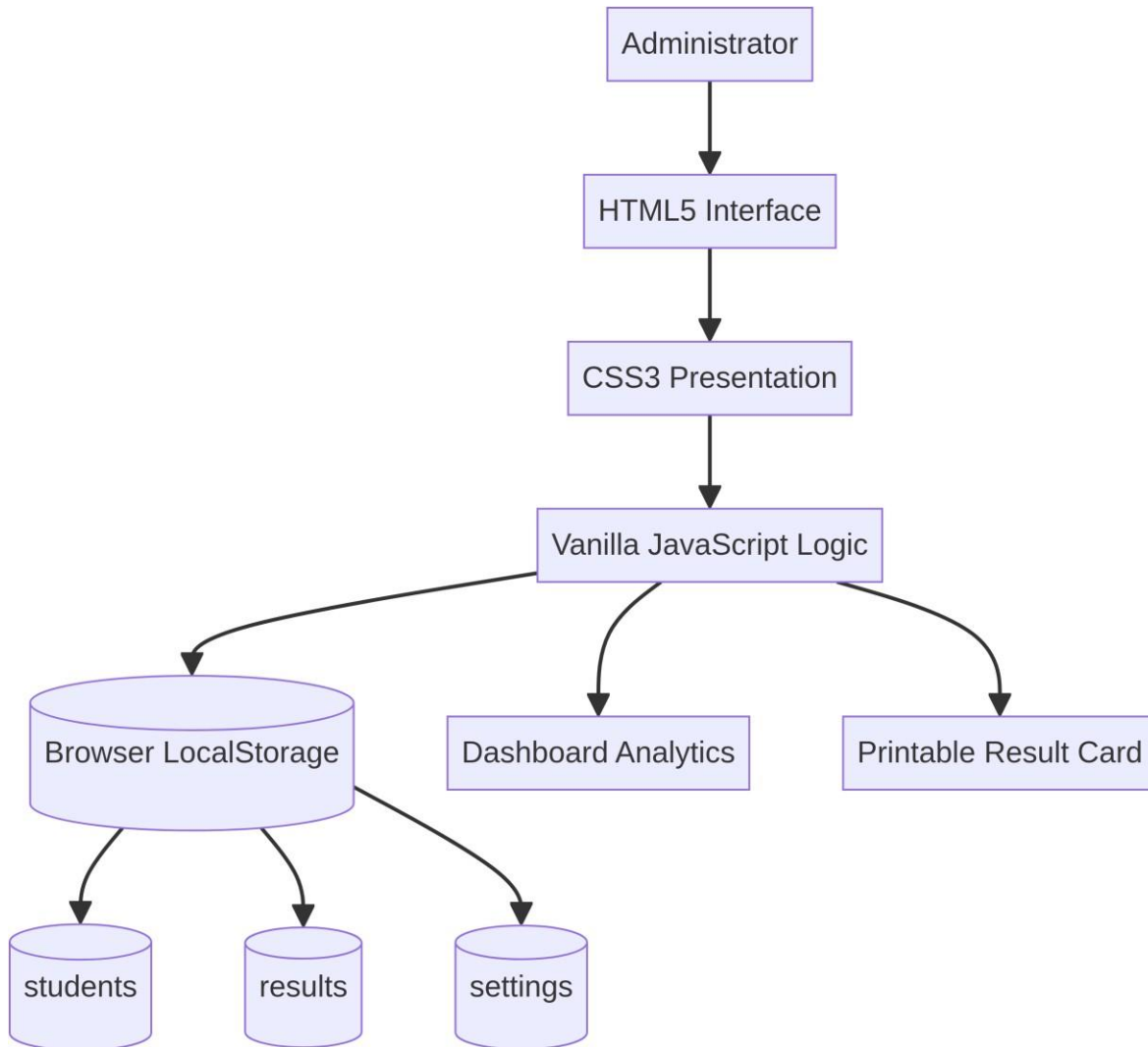


Figure 18: The actual project architecture consists of the administrator, HTML5 interface, CSS3 presentation, JavaScript logic, and browser LocalStorage.

3.2 System Modules

The Login/Welcome module presents the demonstration access form. The Dashboard module calculates live counts, percentages, recent activity, and grade distribution. The Student Management module supports forms,

validation, directory rendering, search, filters, modal details, editing, and deletion. The Result Management module supports student selection, session entry, five subject marks, validation, calculations, CRUD, filtering, and result rendering. The Search module provides a dedicated result-search page. The Result Card module resolves student information, displays subject rows and summary metrics, and invokes browser printing. The LocalStorage module provides shared read, write, add, update, and delete helpers.

3.3 Use-Case Diagram

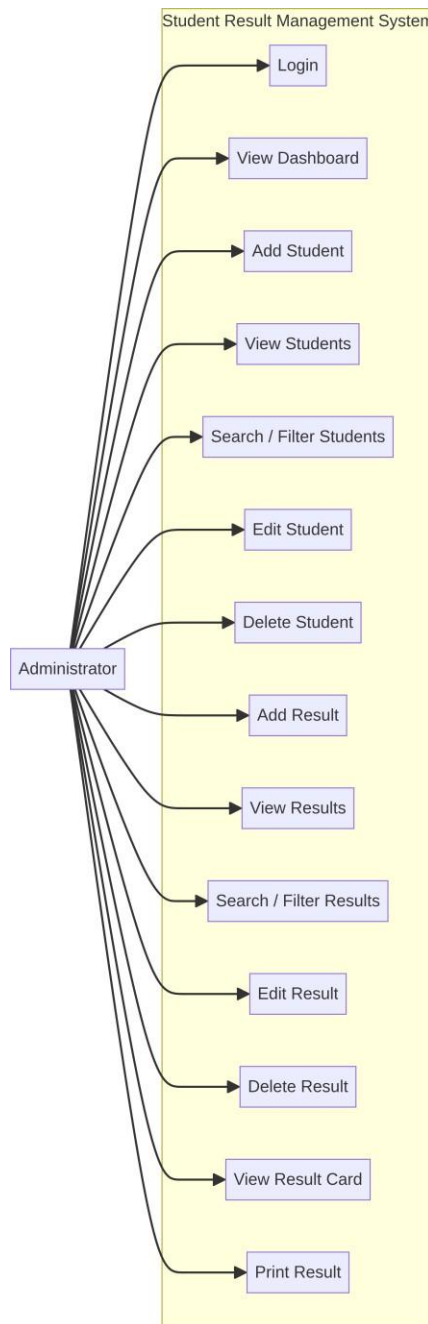


Figure 19: The administrator interacts with the implemented login, dashboard, student, result, search, result-card, and print use cases.

3.4 Data-Flow Diagram

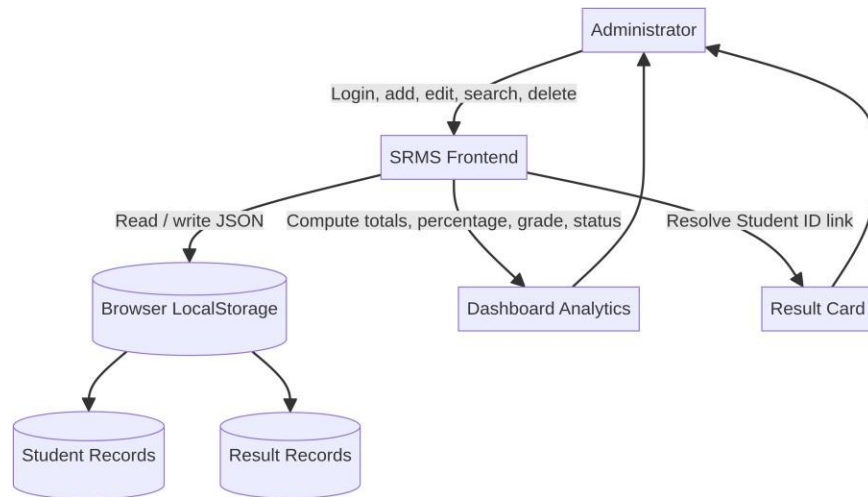


Figure 20: Administrator actions are processed by the SRMS frontend, which reads and writes JSON records in browser LocalStorage and produces dashboard and result-card views.

3.5 Student-Management Flowchart

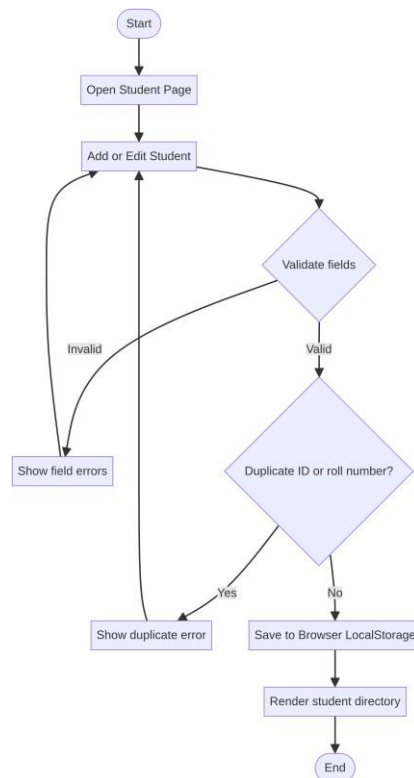


Figure 21: Student data is validated and checked for duplicate identity numbers before it is saved and rendered.

3.6 Result-Management Flowchart

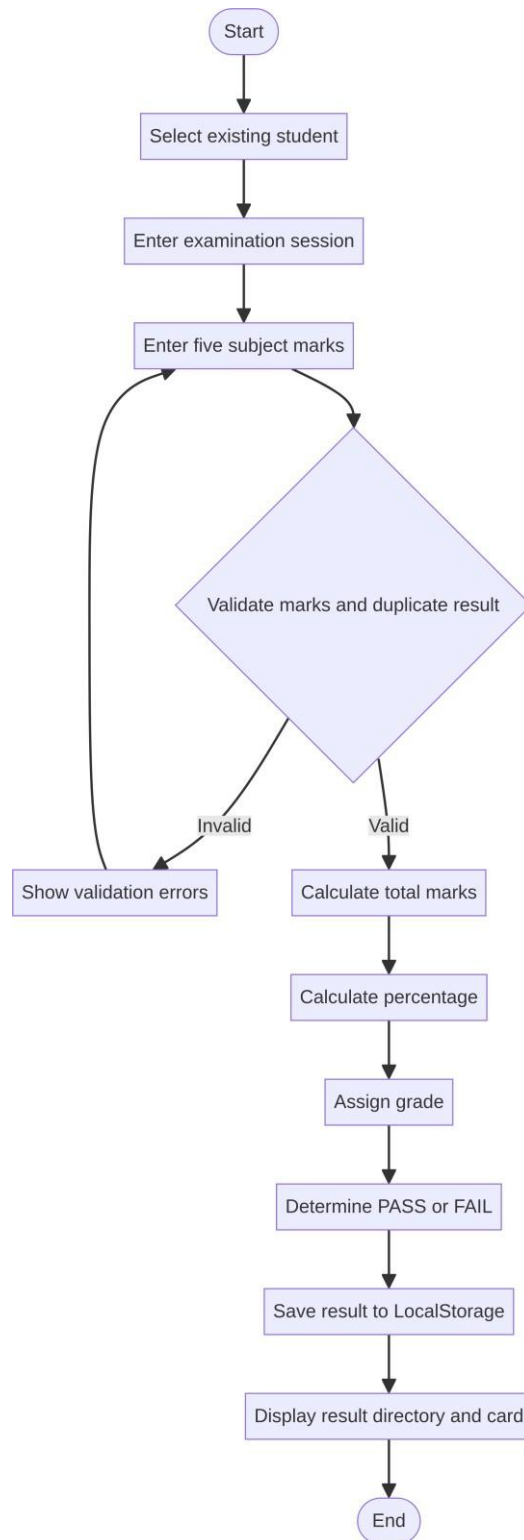


Figure 22: A valid result moves through mark calculation, percentage calculation, grade assignment, status determination, storage, and display.

3.7 Data Storage Design

The project uses `LocalStorage` rather than a database server. The shared storage module serializes arrays and objects with `JSON.stringify()` and reads them back with `JSON.parse()`. The three defined keys are:

Key	Current purpose
<code>`students`</code>	Array of student profile objects
<code>`results`</code>	Array of result objects, each linked with <code>`studentId`</code>
<code>`settings`</code>	Reserved object for settings; the current pages do not use it for a separate settings screen

3.8 Student Data Structure

Field	Description	Demonstration example
<code>`id`</code>	Unique Student ID	<code>`SRMS-1008`</code>
<code>`rollNumber`</code>	Unique roll number	<code>`DIT-008`</code>
<code>`name`</code>	Student name	<code>`Ayesha Khan`</code>
<code>`fatherName`</code>	Father's name	<code>`Imran Khan`</code>
<code>`className`</code>	Class or programme	<code>`DIT`</code>
<code>`section`</code>	Section label	<code>`A`</code>
<code>`dateOfBirth`</code>	ISO-style date string	<code>`2004-05-12`</code>
<code>`gender`</code>	Selected gender value	<code>`Female`</code>
<code>`phone`</code>	Pakistani mobile number	<code>`03001234567`</code>

3.9 Result Data Structure

Field	Description	Demonstration example
<code>`id`</code>	Unique result identifier	<code>`RES-2026-008`</code>
<code>`studentId`</code>	Link to the student record	<code>`SRMS-1008`</code>
<code>`session`</code>	Examination session	<code>`2026`</code>
<code>`subjects`</code>	Array of five subject objects	English, Mathematics, Computer, Physics, Urdu
<code>`subjects[].name`</code>	Subject name	<code>`English`</code>
<code>`subjects[].maxMarks`</code>	Maximum subject marks	<code>`100`</code>
<code>`subjects[].obtainedMarks`</code>	Marks entered by the user	<code>`92`</code>

Total marks, percentage, grade, and status are computed from the stored subject marks rather than stored as independent source fields.

3.10 Student–Result Relationship

The `studentId` in a result is matched against the `id` in the student array. This is a logical application relationship rather than a database foreign-key constraint enforced by a server.

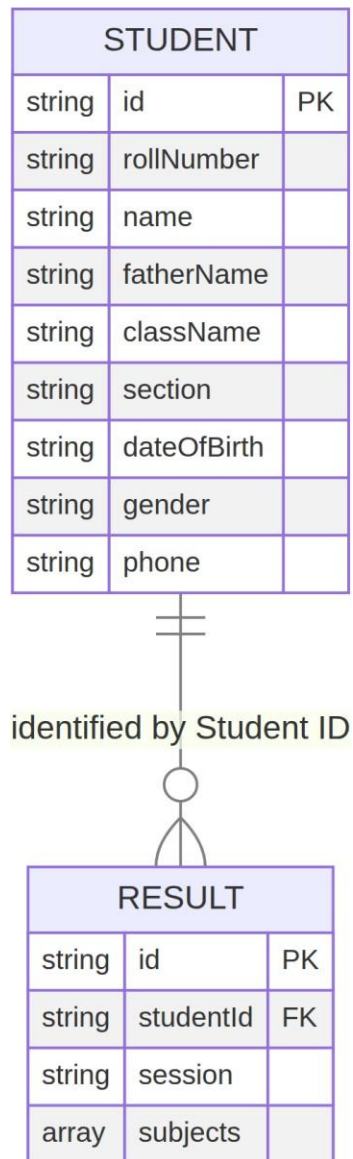


Figure 23: One student can be associated with zero or more result records through the Student ID value.

Chapter 4 — Technologies Used

4.1 HTML5

HTML5 defines the pages, headings, forms, tables, buttons, links, labels, and accessible landmarks. The project uses separate HTML pages for the welcome screen, dashboard, student directory, student form, result directory, result form, result card, and dedicated result search.

4.2 CSS3

CSS3 provides the navy sidebar, cards, statistics panels, badges, forms, tables, modal details, responsive breakpoints, empty states, and print-specific rules. The design uses a shared `css/style.css` file so that the pages maintain a consistent visual language.

4.3 Vanilla JavaScript

Vanilla JavaScript is used without React, Vue, Angular, or a chart library. The modules manipulate the DOM, listen for input and click events, validate form fields, filter arrays, perform calculations, render table rows, show toast messages, and invoke browser printing.

4.4 LocalStorage

LocalStorage is a browser storage mechanism that stores key/value data for a document origin. The project uses it because the dataset is small and the learning objective is to demonstrate client-side persistence. Arrays and objects are converted to JSON when saved and parsed when read. The storage remains available after refresh in the same browser profile, but it is not a secure or shared database.

4.5 Browser Execution

The pages execute directly in a modern browser. A local static server was used during testing so that all pages and assets could be loaded consistently. No API request or backend service is required by the current source code.

4.6 Development Environment

The project source consists of plain HTML, CSS, JavaScript, and image/documentation assets. The implementation does not include a package-managed frontend framework or a Node.js application. Browser-based inspection and local static serving were used for verification.

Chapter 5 — Implementation

5.1 Project Folder Structure

```
student-result-management/  
├── index.html  
├── dashboard.html  
├── students.html  
├── add-student.html  
├── results.html  
├── add-result.html  
├── view-result.html  
├── search-result.html  
├── css/  
│   └── style.css  
├── js/  
│   ├── app.js  
│   ├── storage.js  
│   ├── students.js  
│   ├── results.js  
│   └── dashboard.js  
└── assets/
```

`index.html` contains the welcome and demo-login interface. `dashboard.html` contains statistics, recent records, grade distribution, and quick actions. `students.html` contains the student directory. `add-student.html` contains the add/edit form. `results.html` contains the results directory and filters. `add-result.html` contains the five-subject result form. `view-result.html` contains the result card and print action. `search-result.html` provides a dedicated result-search view. `storage.js` is the shared LocalStorage layer. `students.js` owns student behavior. `results.js` owns result behavior and calculations. `dashboard.js` owns Phase 5 analytics. `app.js` owns shared navigation, login, toast, and print behavior. `style.css` owns the shared design system.

5.2 Login/Welcome Interface

The welcome page presents SRMS branding, a short project description, username and password inputs, and a clear demonstration-access note. The accepted credentials are deliberately visible in the interface because this is a demonstration login, not a secure authentication service.

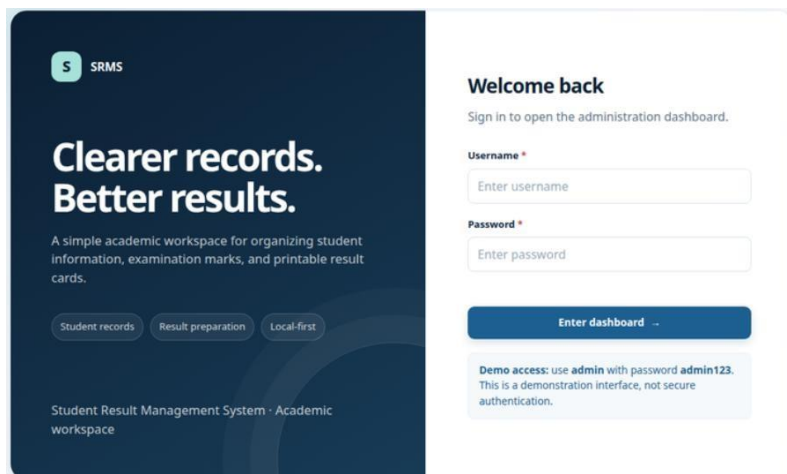


Figure 1: The actual SRMS welcome page displays the login form and clearly identifies the demonstration credentials and security limitation.

5.3 Dashboard Implementation

The dashboard reads the current students and results arrays. It calculates total students, total results, passed results, failed results, pass percentage, and average percentage. It renders the latest four students and latest four results. The grade distribution creates six horizontal bars for A+, A, B, C, D, and F. Empty states are shown when records do not exist.

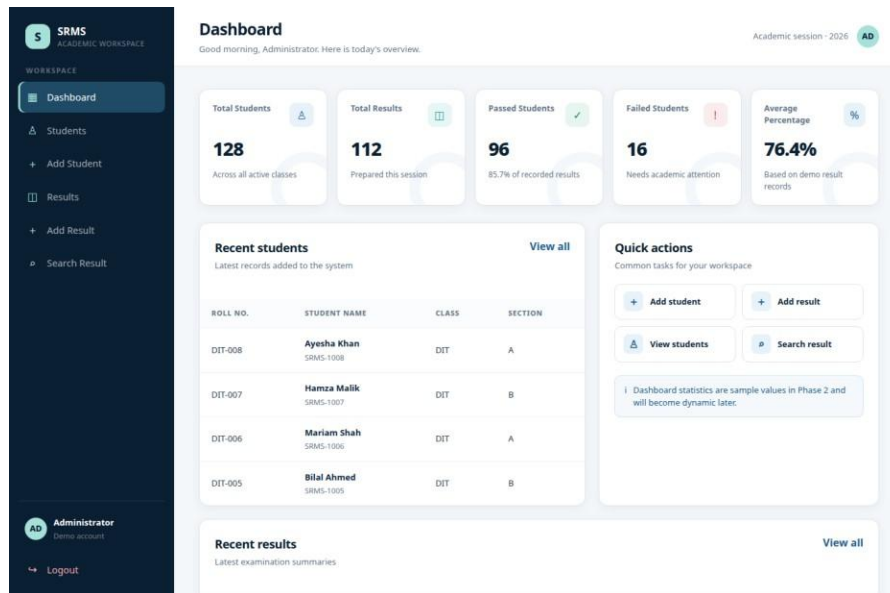


Figure 2: The actual dashboard shows dynamic statistics, recent students, quick actions, grade distribution, and recent results.

5.4 Student Management

The student directory displays Student ID, roll number, name, father name, class, section, and action controls. Search is performed against name, roll number, and Student ID. Class and section filters can be combined with the search query. View opens a modal containing the student's profile. Edit loads the form with existing values and protects the Student ID and roll number from changes. Delete requires confirmation before the LocalStorage record is removed.

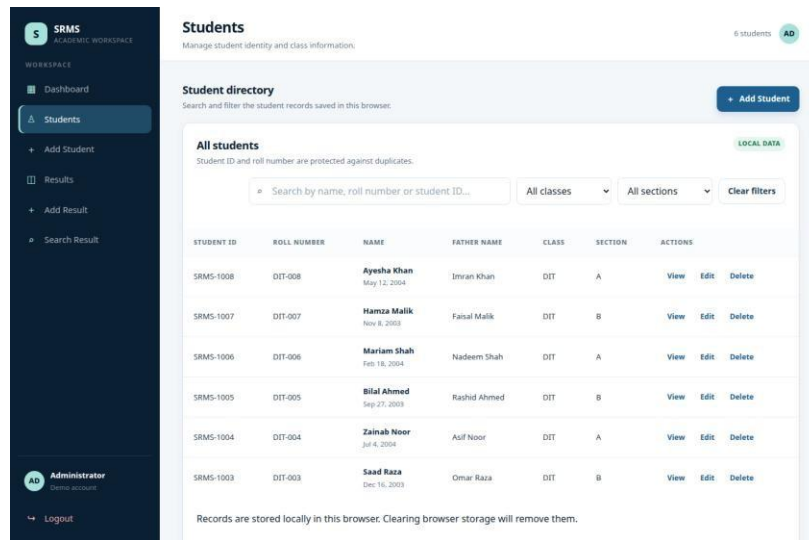


Figure 3: The student directory renders the LocalStorage-backed student records and provides view, edit, and delete controls.

5.5 Add Student Form

The form contains Student ID, Roll Number, Student Name, Father Name, Class, Section, Date of Birth, Gender, and Phone. Required fields are checked. Name fields must contain valid alphabetic name characters, the date cannot be in the future, and the phone must match the implemented Pakistani mobile-number pattern. Student ID and roll number must be unique.

Figure 4: The actual Add Student interface groups identity and additional information fields in the shared SRMS form design.

5.6 Student Validation

Validation errors are displayed beside the related fields. Empty required fields, invalid names, invalid phone numbers, future dates, duplicate Student IDs, and duplicate roll numbers are rejected before the record is stored. The browser's native required-field behavior also supports the HTML form foundation.

5.7 Result Management

The result form first selects an existing student and an examination session. The selected student's name, ID, roll number, class, and section are displayed in linked read-only fields. The marks table contains English, Mathematics, Computer, Physics, and Urdu. Each mark is constrained to the range 0 to 100. A result is rejected when a student is missing, the session format is invalid, marks are empty or outside the range, or the same student already has a result for the same session.

RESULT ID	ROLL NUMBER	STUDENT NAME	SESSION	TOTAL MARKS	OBTAINED	PERCENTAGE	GRADE	STATUS
RES-2026-008	Not linked	Unknown student Session 2026	500	456	91.20%	A+	PASS	View Edit D
RES-2026-007	Not linked	Unknown student Session 2026	500	393	78.60%	B	PASS	View Edit D
RES-2026-006	Not linked	Unknown student Session 2026	500	420	84%	A	PASS	View Edit D
RES-2026-005	Not linked	Unknown student Session 2026	500	239	47.80%	F	FAIL	View Edit D
RES-2026-004	Not linked	Unknown student Session 2026	500	377	75.40%	B	PASS	View Edit D
RES-2026-003	Not linked	Unknown student Session 2026	500	332	66.40%	C	PASS	View Edit D

Figure 5: The results directory displays computed totals, percentages, grades, statuses, and result actions.

Figure 6: The actual Add Result interface provides student selection, five subjects, mark inputs, and a live summary.

5.8 Result Calculation

The implemented formulas are:

Total = English + Mathematics + Computer + Physics + Urdu
 Percentage = Obtained Marks / 500 × 100

The grade rules are:

Percentage	Grade
90–100	A+
80–89.99	A
70–79.99	B
60–69.99	C
50–59.99	D
Below 50	F

The PASS rule is:

PASS = percentage >= 50 AND every subject mark >= 40

Otherwise the result is 'FAIL'. This means a student can have a percentage above 50 and still fail if one subject mark is below 40.

5.9 Live Result Preview

While marks are entered, the result form updates the obtained marks and percentage. The grade and status are shown once all five subjects have values. This gives immediate feedback without saving incomplete data.

5.10 Result Card

The result-card page displays the institute heading, student identity information, session, five subject rows, total marks, obtained marks, percentage, grade, and status. The Print Result button invokes `window.print()`. The current project does not generate or download a PDF itself.

View Result
Preview a professional result card before printing.

Result: RES-2026-008

Northbridge Technical Institute
Department of Information Technology

STUDENT RESULT CARD
Academic Session - 2026

Student name: **Unknown student** | Father name: **Not provided** | Roll number: **Not provided**
Student ID: **SRMS-1008** | Class: **Not provided** | Section: **Not provided**

SUBJECT	MAXIMUM MARKS	OBTAINED MARKS	GRADE
English	100	92	A+
Mathematics	100	89	A
Computer	100	96	A+
Physics	100	86	A
Urdu	100	93	A+

Total marks: **500** | Obtained marks: **456** | Percentage: **91.20%** | Grade: **A+** | Result status: **PASS**

Figure 7: The actual result-card view presents the linked student profile, subject marks, calculated summary, and Print Result action.

5.11 Search Result

The dedicated Search Result page searches stored result records by student name, Student ID, roll number, or result ID. It updates the displayed rows as the query changes and provides View and Edit actions for matches.

Search Result

AD

Find a result

LOCAL DATA

Search by student name, roll number, Student ID, or result ID.

e.g. Ayesha Khan, DIT-008 or SRI

Search

Clear

Matching results

Search results are resolved against the current browser data.

STUDENT	ROLL NUMBER	CLASS
RES-2026-008	DIT-008	Ayesha Khan Session 2026
RES-2026-007	DIT-007	Hamza Malik Session 2026
RES-2026-006	DIT-006	Mariam Shah Session 2026
-----	-----	Bilal Ahmed

Figure 8: The actual Search Result interface adapts to a narrow screen and displays the local-data search workflow.

5.12 Dashboard Analytics

Dashboard analytics are calculated from the same results used by the result directory. The clean demonstration dataset contains six results: five PASS and one FAIL. The grade counts are A+ 1, A 1, B 2, C 1, D 0, and F 1. The dashboard does not use a chart library; the bars are simple CSS elements whose widths are calculated in JavaScript.

5.13 Responsive Design

The responsive stylesheet collapses the sidebar into a menu interaction on narrow screens, reduces the statistics grid, stacks dashboard columns, adapts form controls, and keeps wide tables inside horizontal scroll containers. Genuine desktop and mobile snapshots were captured from the project.

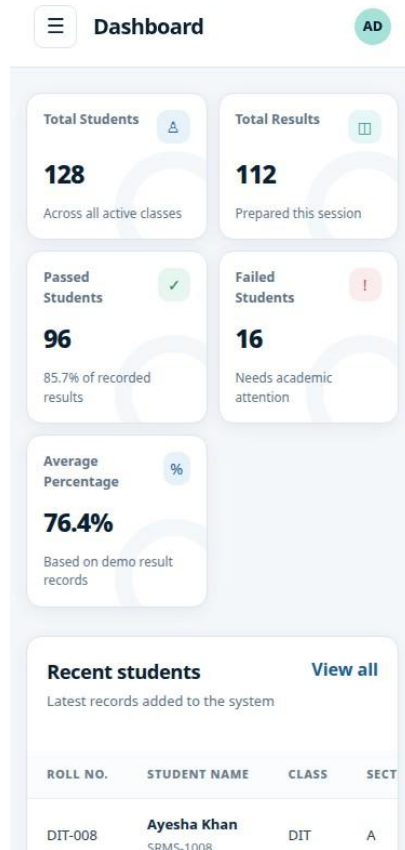


Figure 9: The mobile dashboard retains the statistics, grade distribution, and navigation structure in a narrow viewport.

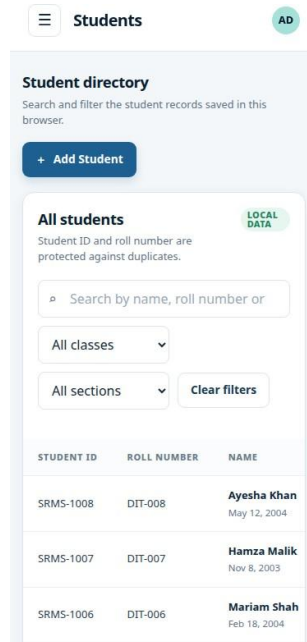
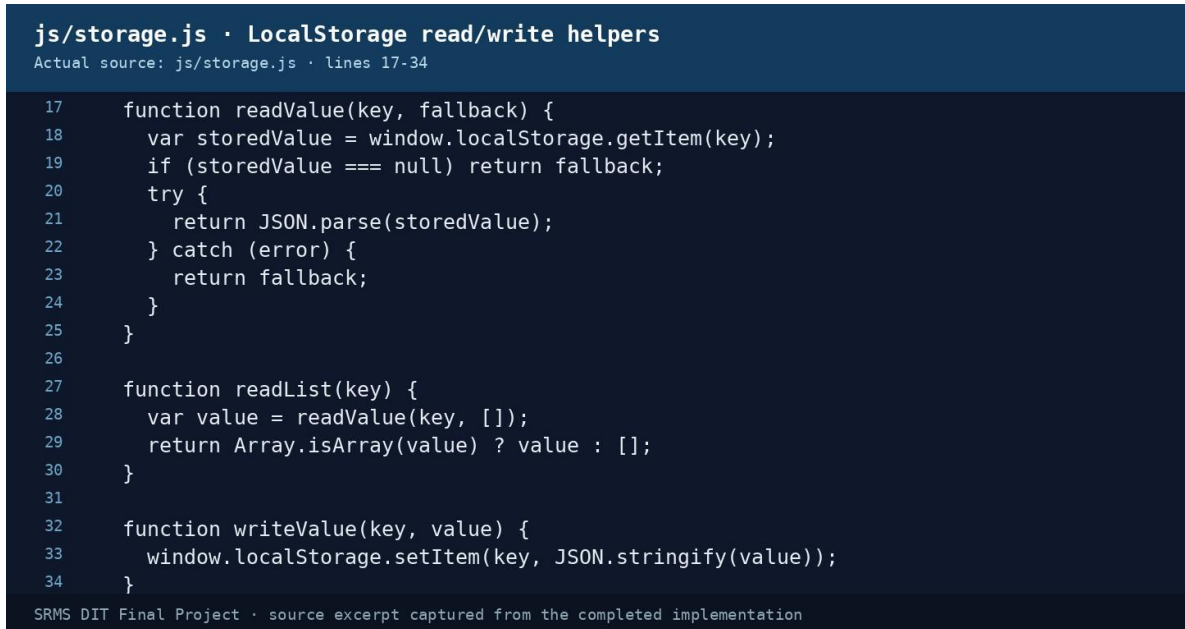


Figure 10: The mobile student directory uses the compact navigation and contained table layout.

Chapter 6 — Source Code Explanation

6.1 LocalStorage Code



```
js/storage.js · LocalStorage read/write helpers
Actual source: js/storage.js · lines 17-34

17  function readValue(key, fallback) {
18      var storedValue = window.localStorage.getItem(key);
19      if (storedValue === null) return fallback;
20      try {
21          return JSON.parse(storedValue);
22      } catch (error) {
23          return fallback;
24      }
25  }
26
27  function readList(key) {
28      var value = readValue(key, []);
29      return Array.isArray(value) ? value : [];
30  }
31
32  function writeValue(key, value) {
33      window.localStorage.setItem(key, JSON.stringify(value));
34  }
```

SRMS DIT Final Project · source excerpt captured from the completed implementation

Figure 11: Actual `js/storage.js` source showing JSON parsing and LocalStorage reading.

The storage helper reads a value by key, returns a fallback when no value exists, and catches invalid JSON. The write helper serializes the JavaScript value before saving it. Higher-level methods use the same functions for students and results, which avoids duplicating storage logic.

6.2 Student Validation Code

```
js/students.js · Student validation and duplicate prevention
Actual source: js/students.js · lines 73-127

73 function validateStudent(student, students, editingId) {
74   var errors = {};
75   var namePattern = /^[A-Za-zĀ-Ķ][A-Za-zĀ-Ķ .-]*$/;
76   var phonePattern = /^(?:\+92|0)3\d{9}$/;
77   var today = new Date().toISOString().slice(0, 10);
78
79   if (!student.id) {
80     errors.studentId = "Student ID is required.";
81   }
82   if (!student.rollNumber) {
83     errors.rollNumber = "Roll number is required.";
84   }
85   if (!student.name) {
86     errors.name = "Student name is required.";
87   } else if (!namePattern.test(student.name) || student.name.replace(/^[A-Za-zĀ-Ķ]/g, "").length < 2) {
88     errors.name = "Please enter a valid student name.";
89   }
90   if (!student.fatherName) {
91     errors.fatherName = "Father name is required.";
92   } else if (!namePattern.test(student.fatherName) || student.fatherName.replace(/^[A-Za-zĀ-Ķ]/g, "").length < 2) {
93     errors.fatherName = "Please enter a valid father name.";
94   }
95   if (!student.className) {
96     errors.className = "Class is required.";
97   }
98   if (!student.section) {
99     errors.section = "Section is required.";
100  }
101  if (!student.dateOfBirth) {
102    errors.dateOfBirth = "Date of birth is required.";
103  } else if (student.dateOfBirth > today || Number.isNaN(new Date(student.dateOfBirth + "T00:00:00").getTime())) {
104    errors.dateOfBirth = "Please enter a valid date of birth.";
105  }
106  if (!student.gender) {
107    errors.gender = "Gender is required.";
108  }
109  if (!student.phone) {
110    errors.phone = "Phone number is required.";
111  } else if (!phonePattern.test(student.phone)) {
112    errors.phone = "Please enter a valid Pakistani phone number.";
113  }
114
115  if (students.some(function (existingStudent) {
116    return normalize(existingStudent.id) === normalize(student.id) && existingStudent.id !== editingId;
117  })) {
118    errors.studentId = "Student ID already exists.";
119  }
120
121  if (students.some(function (existingStudent) {
122    return normalize(existingStudent.rollNumber) === normalize(student.rollNumber) && existingStudent.id !== editingId;
123  })) {
124    errors.rollNumber = "Roll number already exists.";
125  }
126
127  return errors;
}

SRMS DIT Final Project - source excerpt captured from the completed implementation
```

Figure 12: Actual `js/students.js` source showing required-field, name, date, phone, and duplicate validation.

The function builds an `errors` object. It checks each required value, applies the name and phone patterns, checks date validity, and searches existing records for duplicate identity numbers. The editing ID is excluded so an existing record can be updated without being treated as its own duplicate.

6.3 Student Search Code

```
js/students.js · Search and class/section filtering
Actual source: js/students.js · lines 197-218

197  function getFilteredStudents() {
198      var searchField = document.querySelector("[data-student-search]");
199      var classField = document.querySelector("[data-student-class]");
200      var sectionField = document.querySelector("[data-student-section]");
201      var query = normalize(searchField ? searchField.value : "");
202      var className = classField ? classField.value : "";
203      var section = sectionField ? sectionField.value : "";
204
205      return currentStudents.filter(function (student) {
206          var matchesSearch = !query || [student.name, student.rollNumber, student.id].some(function (value) {
207              return normalize(value).indexOf(query) !== -1;
208          });
209          var matchesClass = !className || student.className === className;
210          var matchesSection = !section || student.section === section;
211          return matchesSearch && matchesClass && matchesSection;
212      });
213  }
214
215  function renderStudentList() {
216      visibleStudents = getFilteredStudents();
217      renderStudentRows(visibleStudents);
218  }

SRMS DIT Final Project · source excerpt captured from the completed implementation
```

Figure 13: Actual `js/students.js` source showing search and class/section filter logic.

The query is normalized to lowercase. The code checks the student's name, roll number, and ID. The class and section conditions are combined with the search condition, so all active filters must match.

6.4 Result Calculation Code

```
js/results.js · Total, percentage, grade, and PASS/FAIL
Actual source: js/results.js · lines 40-74

40  function calculateTotal(resultOrMarks) {
41      var marks = Array.isArray(resultOrMarks) ? resultOrMarks : getSubjectMarks(resultOrMarks);
42      return marks.reduce(function (total, mark) { return total + (Number(mark) || 0); }, 0);
43  }
44
45  function calculatePercentage(obtainedMarks) {
46      return Number(((Number(obtainedMarks) / TOTAL_MAX_MARKS) * 100).toFixed(2));
47  }
48
49  function calculateGrade(percentage) {
50      var value = Number(percentage);
51      if (value >= 90) return "A+";
52      if (value >= 80) return "A";
53      if (value >= 70) return "B";
54      if (value >= 60) return "C";
55      if (value >= 50) return "D";
56      return "F";
57  }
58
59  function calculateResultStatus(percentage, marks) {
60      var everySubjectPassed = marks.every(function (mark) { return Number(mark) >= 40; });
61      return Number(percentage) >= 50 && everySubjectPassed ? "PASS" : "FAIL";
62  }
63
64  function getComputedResult(result) {
65      var marks = getSubjectMarks(result);
66      var obtainedMarks = calculateTotal(marks);
67      var percentage = calculatePercentage(obtainedMarks);
68      return {
69          totalMarks: TOTAL_MAX_MARKS,
70          obtainedMarks: obtainedMarks,
71          percentage: percentage,
72          grade: calculateGrade(percentage),
73          status: calculateResultStatus(percentage, marks)
74      };
75  }

SRMS DIT Final Project · source excerpt captured from the completed implementation
```

Figure 14: Actual `js/results.js` source showing total, percentage, grade, and PASS/FAIL functions.

The calculation module defines the five subjects and a total maximum of 500. It sums the subject marks, calculates a two-decimal percentage, applies the grade bands, and checks both the percentage threshold and every-subject threshold.

6.5 Result CRUD Code

```
js/results.js · Result save/update flow
Actual source: js/results.js · lines 251-268

251  form.addEventListener("submit", function (event) {
252      event.preventDefault();
253      clearErrors(form);
254      var result = collectResultFromForm(form);
255      var errors = validateResult(result, window.SRMSStorage.getResults(), form.getAttribute("data-editing-id"));
256      if (Object.keys(errors).length) {
257          showErrors(form, errors);
258          var firstError = form.querySelector(".has-error");
259          if (firstError) firstError.focus();
260          return;
261      }
262      var saved = form.getAttribute("data-editing-id") ? window.SRMSStorage.updateResult(result.id, result) : window.SRMSStorage.addResult(result);
263      if (!saved) return;
264      var computed = getComputedResult(result);
265      window.sessionStorage.setItem("srmsResultMessage", form.getAttribute("data-editing-id") ? "Result updated successfully." : "Result saved successfully.");
266      window.sessionStorage.setItem("srmsResultPreview", JSON.stringify(computed));
267      window.location.href = "results.html";
268  });

SRMS DIT Final Project · source excerpt captured from the completed implementation
```

Figure 15: Actual `js/results.js` source showing validation and save/update behavior.

The submit handler collects the form, clears old errors, validates the result, and calls either `updateResult()` or `addResult()` depending on whether an editing ID exists. It stores a short session message and redirects to the results directory after a successful save.

6.6 Dashboard Statistics Code

```
js/dashboard.js · Dashboard statistics
Actual source: js/dashboard.js · Lines 35-81

35 }
36
37 function calculateFailCount(results) {
38   return results.filter(function (result) { return getComputedResult(result).status === "FAIL"; }).length;
39 }
40
41 function calculatePassPercentage(passCount, totalResults) {
42   return totalResults ? (passCount / totalResults) * 100 : 0;
43 }
44
45 function calculateGradeDistribution(results) {
46   var distribution = {};
47   GRADES.forEach(function (grade) { distribution[grade] = 0; });
48   results.forEach(function (result) {
49     var grade = getComputedResult(result).grade;
50     if (distribution[grade] !== undefined) distribution[grade] += 1;
51   });
52   return distribution;
53 }
54
55 function updateDashboardStats(data) {
56   var passCount = calculatePassCount(data.results);
57   var failCount = calculateFailCount(data.results);
58   var totalResults = data.results.length;
59   var average = totalResults ? data.results.reduce(function (sum, result) { return sum + getComputedResult(result).percentage; }, 0) / totalResults : 0;
60   var values = {
61     studentCount: data.students.length,
62     resultCount: totalResults,
63     passCount: passCount,
64     failCount: failCount,
65     passPercentage: formatPercentage(calculatePassPercentage(passCount, totalResults)),
66     averagePercentage: formatPercentage(average)
67   };
68   Object.keys(values).forEach(function (key) {
69     document.querySelectorAll("[data-dashboard-" + key.replace(/[A-Z]/g, function (letter) { return "-" + letter.toLowerCase(); }) + "]").forEach(function (element) {
70       element.textContent = values[key];
71     });
72   });
73   var resultFoot = document.querySelector("[data-dashboard-result-foot]");
74   if (resultFoot) resultFoot.textContent = totalResults ? "Prepared from LocalStorage records" : "Add a result to begin tracking";
75   var passFoot = document.querySelector("[data-dashboard-pass-foot]");
76   if (passFoot) passFoot.textContent = totalResults ? values.passPercentage + " of recorded results" : "No results recorded yet";
77   var failFoot = document.querySelector("[data-dashboard-fail-foot]");
78   if (failFoot) failFoot.textContent = failCount ? "Needs academic attention" : "No failed results recorded";
79   var averageFoot = document.querySelector("[data-dashboard-average-foot]");
80   if (averageFoot) averageFoot.textContent = totalResults ? "Average across recorded results" : "No results recorded yet";
81 }
```

Figure 16: Actual `js/dashboard.js` source showing `LocalStorage` loading and dashboard statistic calculations.

The dashboard obtains the current arrays, computes pass and fail counts using the shared result calculation functions, calculates the pass percentage and average percentage, and writes values into elements with dashboard data attributes.

6.7 Grade Distribution Code

```
js/dashboard.js · CSS bar data generation
Actual source: js/dashboard.js · Lines 111-123

111 function renderGradeDistribution(results) {
112   var distribution = calculateGradeDistribution(results);
113   var list = document.querySelector("[data-grade-distribution]");
114   var empty = document.querySelector("[data-grade-distribution-empty]");
115   if (!list) return;
116   var maximum = Math.max.apply(null, GRADES.map(function (grade) { return distribution[grade]; }).concat([1]));
117   list.innerHTML = GRADES.map(function (grade) {
118     var count = distribution[grade];
119     var width = count ? Math.max((count / maximum) * 100, 6) : 0;
120     return "<li class='distribution-row'><span class='distribution-label'>" + grade + "</span><span class='distribution-track' role='progressbar' style='width:" + width + "%'></span></li>";
121   }).join("");
122   if (empty) empty.hidden = Boolean(results.length);
123 }
```

Figure 17: Actual `js/dashboard.js` source showing grade distribution and CSS-bar generation.

The distribution function initializes all six grade categories, counts each computed result, finds the largest count, and assigns a proportional bar width. A zero-result case is handled separately by showing the grade-distribution empty state.

Chapter 7 — User Interface and Screenshots

This chapter records screenshots captured from the actual SRMS implementation. The desktop images in the project test set were generated at 1366×900 and the mobile images at 375×800. The screenshots are not mockups; they show the running pages and their real stored demonstration records.

Figure	Actual evidence included
1	Welcome/login page captured from the running application
2	Dashboard with dynamic Phase 5 statistics
3	Student directory
4	Add Student interface
5	Results directory
6	Add Result interface
7	Result card and browser-print layout
8	Search Result page on a narrow viewport
9	Mobile dashboard
10	Mobile student directory
11–17	Real source-code excerpts rendered from the project files
18–23	Deterministic diagrams based on the actual source and data relationship

Separate validation-error and edit-state images were not included in this final image set because the available captured assets did not contain a clean, named screenshot for each state. The corresponding behaviors were tested through browser interaction and are documented in Chapter 8 rather than represented by fabricated images.

Chapter 8 — Testing and Validation

8.1 Testing Introduction

Testing was performed through browser loading, browser interaction, LocalStorage inspection, structural source checks, and responsive screenshots. Temporary records used for CRUD testing were removed after verification so that the clean demonstration dataset was restored.

8.2 Testing Strategy

The strategy combined functional testing of forms and actions, validation testing for invalid values, calculation testing using known marks, persistence testing across refreshes, responsive snapshot testing, and structural checks for links, controls, and prohibited technologies. The testing record is stored in `testing-notes.md`, `testing-validation.txt`, and `phase5-verification.md`.

8.3 Functional Test Cases

ID	Feature	Input/action	Expected result	Actual result	Status
T-01	Login	`admin` / `admin123`	Dashboard opens	Dashboard opened	Pass
T-02	Add student	Valid nine-field profile	One record is added	Count increased and record appeared	Pass
T-03	Edit student	Change name of existing record	Same ID remains with updated value	Updated value appeared, no duplicate	Pass
T-04	Delete student	Confirm delete	Record is removed	Record absent after refresh	Pass
T-05	Search student	Search `SRMS- 1003`	One matching row	Saad Raza row returned	Pass
T-06	Filter student	Class DIT and section A	Matching rows remain	Combined filtering worked	Pass
T-07	Add result	Valid student, session, and marks	Result is stored	Result directory updated	Pass
T-08	Edit result	Change one mark	Same result is recalculated	Obtained marks and percentage	Pass

				changed	
T-09	Delete result	Confirm delete	Result is removed	Count returned to baseline	Pass
T-10	Search result	Search name, ID, or roll number	Matching result appears	Queries returned expected record	Pass
T-11	Calculation	80, 90, 70, 60, 100	400, 80%, A, PASS	Exact values displayed	Pass
T-12	PASS/FAIL	One subject 35, other subjects 80	FAIL despite percentage above 50	FAIL displayed	Pass
T-13	Dashboard	Load clean dataset	Dynamic summary appears	6, 6, 5, 1, 83.33%, 73.90%	Pass
T-14	Print	Click Print Result	Browser print action invoked	Print handler executed	Pass

8.4 Validation Testing

Validation case	Expected behavior	Observed result
Empty student fields	Required-field messages or native validity	Rejected
Duplicate Student ID	Clear duplicate error	Rejected
Duplicate roll number	Clear duplicate error	Rejected
Invalid phone	Pakistani phone format error	Rejected
Invalid student name	Name validation error	Rejected
Invalid marks	Marks must be 0–100	Rejected
Marks below 0	Invalid mark error	Rejected
Marks above 100	Invalid mark error	Rejected
Invalid student selection	Existing student is required	Rejected
Duplicate result	Same student/session combination rejected	Rejected

8.5 Boundary Testing

Boundary input	Interpretation in the implementation
----------------	--------------------------------------

0 marks	Valid numeric input but normally produces F and FAIL.
40 in every subject	Satisfies the subject threshold; percentage is 40%, so status is FAIL.
49 in every subject	Percentage is 49%, so grade F and status FAIL.
50% with every subject ≥ 40	Meets both conditions and produces PASS.
100 in every subject	Produces 500/500, 100%, grade A+, and PASS.

8.6 Persistence Testing

The LocalStorage round-trip test saved a temporary array with JSON serialization, read it back, confirmed the values, and removed the temporary record. Student and result CRUD tests also confirmed that changes remained after refresh. The clean demo state was restored after temporary tests.

8.7 Responsive Testing

Viewport	Observed behavior
1366×900	Fixed sidebar, multi-column dashboard, readable tables, no obvious horizontal overflow in the main layout.
768×900	Compressed dashboard and form layout with contained wide tables.
375×800	Menu-based sidebar, stacked content, two-column compact statistics, and horizontally scrollable tables inside their containers.

8.8 Regression Testing

After student and result modules were completed, the earlier navigation, shared styling, LocalStorage helpers, page links, and print action were rechecked. Phase 5 dashboard testing also verified that student and result CRUD changes changed the dashboard values after re-render and refresh.

8.9 Test Summary

The tested workflows passed for the clean demonstration dataset. Structural validation reported all eight HTML pages passing, zero prohibited-technology matches, zero inline event-handler attributes, and valid local asset references. The tests support the documented functionality but do not constitute production security or institutional acceptance testing.

Chapter 9 — Results and Discussion

9.1 Final System Outcome

The final SRMS provides a connected browser workspace for student records and examination results. Its modules share LocalStorage helpers and use Student ID to connect result records to students.

9.2 Student Management Results

Student CRUD was verified using the directory and form pages. Search by name, Student ID, and roll number returned expected rows. Class and section filters combined with search. Duplicate identity values were prevented, and deletion was confirmed and persistent.

9.3 Result Management Results

The result form linked existing students to five-subject result records. Invalid marks were rejected, duplicate student/session results were prevented, and edit operations recalculated the same result record without creating a duplicate.

9.4 Dashboard Results

The clean demo dataset produced six students, six results, five passes, one failure, an 83.33% pass percentage, and a 73.90% average percentage. The grade distribution showed one A+, one A, two B grades, one C, zero D grades, and one F grade.

9.5 Search and Filtering Results

Student and result searches were tested using names, Student IDs, and roll numbers. Student class and section filters and result session, grade, and status filters were included in the tested workflows.

9.6 Calculation Accuracy

The known calculation case of 80, 90, 70, 60, and 100 produced 400 obtained marks, 80%, grade A, and PASS. The threshold case with one mark of 35 produced 71%, grade B, and FAIL, confirming that the subject-level minimum is applied independently of the overall percentage.

9.7 User Interface Evaluation

The shared design system creates consistent navigation, cards, badges, forms, tables, empty states, and action links. It provides enough visual hierarchy for an administrator to understand the main tasks without a complex component framework.

9.8 Responsive Behavior

The desktop, tablet, and mobile snapshots show that the application remains usable across common viewport sizes. Tables are contained in scroll regions rather than forcing the entire page to overflow.

9.9 System Limitations

The system is limited by client-side storage, demonstration authentication, single-browser data ownership, lack of concurrent users, and the absence of backend validation. A browser user with access to the profile can inspect or modify LocalStorage records.

9.10 Overall Discussion

Within its educational scope, the project successfully demonstrates an end-to-end frontend application. It does not attempt to replace a secure institutional system. Its strongest value is that the data model, page structure, calculations, CRUD operations, analytics, and testing evidence are small enough to understand and explain.

Chapter 10 — Conclusion, Security, Limitations and Future Enhancements

10.1 Security Considerations

The login is a fixed demonstration check in client-side JavaScript and is not secure authentication. LocalStorage is also client-side, so the browser user can inspect, change, or clear the stored values. There is no server-side access control, authorization, audit log, encryption layer, or secure session service in the current project.

10.2 Current Limitations and Future Enhancements

Current limitation	Possible future enhancement
No backend	Add a backend API for controlled data access.
No database server	Use MySQL, MongoDB, or another server database.
No secure authentication	Add hashed credentials and secure sessions.
No multi-user support	Add administrator, teacher, and student roles.
Browser-dependent LocalStorage	Add cloud storage and synchronization.
No email delivery	Add result notifications through a secure service.
No built-in PDF generation	Add server-side or client-side PDF export.
No backup/restore workflow	Add validated export and import files.
Basic dashboard analytics	Add date, class, subject, and session reports.
Five fixed subjects	Add configurable classes and subject lists.
No attendance module	Integrate attendance records and reports.
No student portal	Add controlled student and parent access.

All features in the right-hand column are **future enhancements**, not current functionality.

Conclusion

The Student Result Management System was developed as a DIT final-semester project using HTML5, CSS3, Vanilla JavaScript, and browser LocalStorage. The completed application manages student records, validates and persists student information, accepts five-subject examination marks, calculates total marks and percentage, assigns grades, determines PASS/FAIL status, provides search and filters, displays dashboard analytics, renders a linked result card, and invokes browser printing.

The project addresses common record-management and manual-calculation problems within a small educational scope. Testing confirmed the main CRUD workflows, validation rules, calculation rules, dashboard values, LocalStorage persistence, empty states, search behavior, result linking, and responsive

layouts. The implementation also helped demonstrate modular JavaScript, DOM events, reusable storage helpers, data validation, and user-interface design.

The system remains a prototype rather than a production institutional platform. It has no backend, database server, secure authentication, multi-user environment, cloud synchronization, email delivery, or built-in PDF generation. These limitations are clearly identified and provide realistic opportunities for future work. Overall, the project meets its educational objectives and provides a functional foundation that could later be extended with secure server-side services.

References

- [1]: <https://developer.mozilla.org/en-US/docs/Web/HTML> "MDN Web Docs: HTML"
- [2]: <https://developer.mozilla.org/en-US/docs/Web/CSS> "MDN Web Docs: CSS"
- [3]: <https://developer.mozilla.org/en-US/docs/Web/JavaScript> "MDN Web Docs: JavaScript"
- [4]: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API "MDN Web Docs: Web Storage API"
- [5]: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage> "MDN Web Docs: Window.localStorage"
- [6]: <https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener> "MDN Web Docs: EventTarget.addEventListener()"
- [7]: <https://developer.mozilla.org/en-US/docs/Web/API/Window/print> "MDN Web Docs: Window.print()"